

6.4 Advanced Dependency Injection

Part of Services and Dependency Injection Module - Duration: 10-12 hours

⚡ Important:

Standalone Components & Advanced DI:

All Advanced DI techniques work with **both NgModules and Standalone Components**. This guide shows examples using modules for clarity, but every technique can be used in standalone components by specifying providers in:

- Component's `providers` array
- `bootstrapApplication()` for root-level providers
- `providedIn: 'root'` in services
- Route providers in standalone routing

See **Section 9** for complete standalone examples of all techniques.

1. Introduction to Advanced DI

1.1 What is Advanced DI?

While basic dependency injection covers service creation and injection through constructors, advanced DI provides fine-grained control over **how**, **when**, and **what** gets injected. These techniques handle complex scenarios like:

- Creating services with custom logic (factory functions)
- Providing existing values or configurations
- Making dependencies optional
- Controlling injector hierarchy
- Supporting multiple providers for the same token

1.2 When Do You Need Advanced DI?

Scenario	Advanced DI Solution
Service needs configuration at creation	Factory providers
Want to provide a constant value	useValue providers
Need to alias one service to another	useExisting providers
Dependency might not be available	@Optional decorator
Need to skip parent injectors	@SkipSelf decorator
Multiple services for same token	Multi-provider tokens

2. Factory Providers

2.1 What is a Factory Provider?

A factory provider uses a function to create a service instance. This gives you complete control over the service creation process, allowing you to inject dependencies, pass configuration, or implement custom logic.

Why Use Factory Providers?

- **Conditional Creation:** Create different implementations based on conditions
- **Configuration:** Pass settings during service creation
- **Complex Initialization:** Run custom logic before returning service
- **Dependency Injection in Factory:** Factory can request its own dependencies

2.2 Basic Factory Provider Syntax

```
// Basic structure of factory provider
{
  provide: ServiceToken,           // What token to provide for
```

```
useFactory: factoryFunction,    // Function that creates the service
deps: [Dependencies]           // Dependencies the factory needs
}
```

2.3 Simple Factory Example

```
// logger.service.ts
export class LoggerService {
  constructor(private prefix: string) { }

  log(message: string): void {
    console.log(`${this.prefix}: ${message}`);
  }
}

// logger.factory.ts
export function loggerFactory(): LoggerService {
  const environment = 'development'; // Could come from config
  const prefix = environment === 'development' ? '[DEV]' : '[PROD]';
  return new LoggerService(prefix);
}

// app.module.ts
import { NgModule } from '@angular/core';
import { LoggerService } from './logger.service';
import { loggerFactory } from './logger.factory';

@NgModule({
  providers: [
    {
      provide: LoggerService,
      useFactory: loggerFactory
    }
  ]
})
export class AppModule { }
```

Output:

Result:

When LoggerService is injected, the factory function runs:

- Checks environment

- Creates LoggerService with '[DEV]' prefix
- Returns configured instance
- All components get logger with correct prefix

2.4 Factory with Dependencies

Factories can request their own dependencies through the `deps` array:

```
// config.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ConfigService {
  getApiUrl(): string {
    return 'https://api.example.com';
  }

  getTimeout(): number {
    return 5000;
  }

  isDebugMode(): boolean {
    return true;
  }
}
```

```
// http-client.service.ts
export class HttpClientService {
  constructor(
    private apiUrl: string,
    private timeout: number,
    private debug: boolean
  ) {
    console.log('HttpClientService configured:');
    console.log('API URL:', this.apiUrl);
    console.log('Timeout:', this.timeout);
    console.log('Debug:', this.debug);
  }
}
```

```
get(endpoint: string): void {
  const url = `${this.apiUrl}${endpoint}`;
  console.log(`GET request to: ${url}`);
  if (this.debug) {
    console.log(`[DEBUG] Timeout set to ${this.timeout}ms`);
  }
}
```

```
// http-client.factory.ts
import { ConfigService } from './config.service';
import { HttpClientService } from './http-client.service';

export function httpClientFactory(config: ConfigService): HttpClientService {
  return new HttpClientService(
    config.getApiUrl(),
    config.getTimeout(),
    config.isDebugEnabled()
  );
}

// app.module.ts
@NgModule({
  providers: [
    {
      provide: HttpClientService,
      useFactory: httpClientFactory,
      deps: [ConfigService] // Factory receives ConfigService
    }
  ]
})
export class AppModule { }
```

Output:

Factory with Dependencies Flow:

1. Component requests HttpClientService
2. Angular sees factory provider
3. Angular injects ConfigService into factory

4. Factory reads config values
5. Factory creates HttpClientService with config
6. Angular returns configured instance to component

2.5 Conditional Factory Example

Factories can return different implementations based on conditions:

```
// storage.interface.ts
export interface StorageService {
  save(key: string, value: string): void;
  load(key: string): string | null;
}

// local-storage.service.ts
export class LocalStorageService implements StorageService {
  save(key: string, value: string): void {
    localStorage.setItem(key, value);
    console.log('Saved to localStorage');
  }

  load(key: string): string | null {
    return localStorage.getItem(key);
  }
}

// memory-storage.service.ts
export class MemoryStorageService implements StorageService {
  private storage: { [key: string]: string } = {};

  save(key: string, value: string): void {
    this.storage[key] = value;
    console.log('Saved to memory');
  }

  load(key: string): string | null {
    return this.storage[key] || null;
  }
}
```

```
// storage.factory.ts
import { StorageService } from './storage.interface';
import { LocalStorageService } from './local-storage.service';
import { MemoryStorageService } from './memory-storage.service';

export function storageFactory(): StorageService {
  // Check if localStorage is available
  const isLocalStorageAvailable = typeof localStorage !== 'undefined';

  if (isLocalStorageAvailable) {
    console.log('Using LocalStorageService');
    return new LocalStorageService();
  } else {
    console.log('Using MemoryStorageService (fallback)');
    return new MemoryStorageService();
  }
}

// app.module.ts
@NgModule({
  providers: [
    {
      provide: 'StorageService', // Using string token
      useFactory: storageFactory
    }
  ]
})
export class AppModule { }
```

```
// component using the factory-created service
import { Component, Inject } from '@angular/core';
import { StorageService } from './storage.interface';

@Component({
  selector: 'app-data',
  template: `
    <div>
      <h3>Data Storage</h3>
      <button (click)="saveData()">Save Data</button>
      <button (click)="loadData()">Load Data</button>
      <p>{{ message }}</p>
    </div>
  `
})
```

```
export class DataComponent {  
  message: string = '';  
  
  constructor(@Inject('StorageService') private storage: StorageService) { }  
  
  saveData(): void {  
    this.storage.save('username', 'JohnDoe');  
    this.message = 'Data saved!';  
  }  
  
  loadData(): void {  
    const data = this.storage.load('username');  
    this.message = data ? `Loaded: ${data}` : 'No data found';  
  }  
}
```

Output:

Conditional Factory Behavior:

- In browser → Uses LocalStorageService
- In server-side rendering → Uses MemoryStorageService
- Component doesn't know which implementation it gets
- Both implement same interface, so component code works with either

2.6 When to Use Factory Providers

💡 Tip:

Use Factory Providers When:

- ✓ Service needs configuration values at creation
- ✓ Need to choose between implementations at runtime
- ✓ Service creation requires complex initialization logic
- ✓ Want to inject dependencies into service constructor
- ✓ Need to perform checks before service creation

Don't Use When:

- ✗ Simple service with no configuration needed
- ✗ Standard dependency injection works fine
- ✗ No conditional logic required

3. useValue and useExisting Providers

3.1 useValue Provider

What is useValue?

`useValue` provides an existing object, value, or constant directly without creating a new instance. It's useful for configuration objects, constants, or pre-existing instances.

Basic useValue Example

```
// app.config.ts - Configuration object
export const APP_CONFIG = {
  apiUrl: 'https://api.example.com',
  appName: 'My Angular App',
  version: '1.0.0',
  maxRetries: 3,
  timeout: 5000
};

// app.module.ts
import { APP_CONFIG } from './app.config';

@NgModule({
  providers: [
    {
      provide: 'AppConfig', // Token name
      useValue: APP_CONFIG // Provide the actual object
    }
  ]
})
export class AppModule { }
```

```
// Using the configuration in a component
import { Component, Inject } from '@angular/core';

@Component({
  selector: 'app-settings',
  template: `
    <div>
      <h3>{{ getAppName() }}</h3>
      <p>Version: {{ getVersion() }}</p>
      <p>API: {{ getApiUrl() }}</p>
    </div>
  `
})
export class SettingsComponent {

  constructor(@Inject('AppConfig') private config: any) { }

  getAppName(): string {
    return this.config.appName;
  }

  getVersion(): string {
    return this.config.version;
  }

  getApiUrl(): string {
    return this.config.apiUrl;
  }
}
```

Output:

Component Display:

My Angular App

Version: 1.0.0

API: https://api.example.com

useValue with InjectionToken (Type-Safe)

For better type safety, use `InjectionToken` instead of string tokens:

```
// app.config.ts
import { InjectionToken } from '@angular/core';

export interface AppConfig {
  apiUrl: string;
  appName: string;
  version: string;
  maxRetries: number;
  timeout: number;
}

export const APP_CONFIG_TOKEN = new InjectionToken<AppConfig>('app.config');

export const APP_CONFIG: AppConfig = {
  apiUrl: 'https://api.example.com',
  appName: 'My Angular App',
  version: '1.0.0',
  maxRetries: 3,
  timeout: 5000
};

// app.module.ts
@NgModule({
  providers: [
    {
      provide: APP_CONFIG_TOKEN,
      useValue: APP_CONFIG
    }
  ]
})
export class AppModule { }
```

```
// Type-safe usage in component
import { Component, Inject } from '@angular/core';
import { APP_CONFIG_TOKEN, AppConfig } from './app.config';

@Component({
  selector: 'app-settings'
})
export class SettingsComponent {

  // TypeScript knows the type!
  constructor(@Inject(APP_CONFIG_TOKEN) private config: AppConfig) {
```

```
// Autocomplete works: config.apiUrl, config.appName, etc.
}
}
```

useValue for Feature Flags

```
// feature-flags.ts
export interface FeatureFlags {
  enableNewUI: boolean;
  enableBetaFeatures: boolean;
  enableAnalytics: boolean;
}

export const FEATURE_FLAGS_TOKEN = new InjectionToken<FeatureFlags>('feature.flags');

export const PRODUCTION_FLAGS: FeatureFlags = {
  enableNewUI: true,
  enableBetaFeatures: false,
  enableAnalytics: true
};

export const DEVELOPMENT_FLAGS: FeatureFlags = {
  enableNewUI: true,
  enableBetaFeatures: true,
  enableAnalytics: false
};

// app.module.ts
@NgModule({
  providers: [
    {
      provide: FEATURE_FLAGS_TOKEN,
      useValue: DEVELOPMENT_FLAGS // Or PRODUCTION_FLAGS
    }
  ]
})
export class AppModule { }
```

```
// Using feature flags
import { Component, Inject } from '@angular/core';
import { FEATURE_FLAGS_TOKEN, FeatureFlags } from '../feature-flags';

@Component({
```

```
selector: 'app-features',
template: `
  <div>
    <div *ngIf="showNewUI()">
      <h3>New UI Enabled!</h3>
    </div>
    <div *ngIf="showBetaFeatures()">
      <p>Beta features available</p>
    </div>
  </div>
`
})
export class FeaturesComponent {

  constructor(@Inject(FEATURE_FLAGS_TOKEN) private flags: FeatureFlags) { }

  showNewUI(): boolean {
    return this.flags.enableNewUI;
  }

  showBetaFeatures(): boolean {
    return this.flags.enableBetaFeatures;
  }
}
```

3.2 useExisting Provider

What is useExisting?

`useExisting` creates an alias - it tells Angular to provide an existing service when a different token is requested. This is useful for providing the same service instance under multiple names or for backward compatibility.

Basic useExisting Example

```
// logger.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class LoggerService {
  log(message: string): void {
```

```
    console.log('[LOG]', message);
  }

  error(message: string): void {
    console.error('[ERROR]', message);
  }
}

// app.module.ts
@NgModule({
  providers: [
    // Create an alias: when 'Logger' is requested, provide LoggerService
    {
      provide: 'Logger',
      useExisting: LoggerService
    }
  ]
})
export class AppModule { }
```

```
// Both injections get the SAME instance
import { Component, Inject } from '@angular/core';
import { LoggerService } from './logger.service';

@Component({
  selector: 'app-demo'
})
export class DemoComponent {

  constructor(
    private loggerService: LoggerService,    // Original
    @Inject('Logger') private logger: any    // Alias
  ) {
    // Both point to same instance
    console.log(this.loggerService === this.logger); // true
  }
}
```

useExisting for Interface Abstraction

```
// data-storage.interface.ts
export abstract class DataStorageService {
  abstract save(data: any): void;
}
```

```
    abstract load(): any;
  }

  // local-storage.service.ts
  import { Injectable } from '@angular/core';
  import { DataStorageService } from '../data-storage.interface';

  @Injectable({
    providedIn: 'root'
  })
  export class LocalStorageService implements DataStorageService {
    save(data: any): void {
      localStorage.setItem('data', JSON.stringify(data));
      console.log('Data saved to localStorage');
    }

    load(): any {
      const data = localStorage.getItem('data');
      return data ? JSON.parse(data) : null;
    }
  }

  // app.module.ts
  @NgModule({
    providers: [
      // When DataStorageService is requested, use LocalStorageService
      {
        provide: DataStorageService,
        useExisting: LocalStorageService
      }
    ]
  })
  export class AppModule { }
```

```
// Component uses abstract class
import { Component } from '@angular/core';
import { DataStorageService } from '../data-storage.interface';

@Component({
  selector: 'app-data'
})
export class DataComponent {

  // Injected as abstract class
  constructor(private storage: DataStorageService) {
```

```
// Actually receives LocalStorageService instance
}

saveData(): void {
  this.storage.save({ name: 'John', age: 30 });
}
}
```

Output:

useExisting Benefit:

- Component depends on abstract DataStorageService
- Gets concrete LocalStorageService implementation
- Easy to swap implementations (just change provider)
- Component code doesn't need to change

3.3 useValue vs useExisting Comparison

Feature	useValue	useExisting
Purpose	Provide a specific value/object	Alias an existing provider
What it provides	The exact value given	Reference to another service
Instance creation	No creation (value exists)	No creation (references existing)
Common use	Config objects, constants	Service aliases, abstraction
Type safety	Use InjectionToken	Uses class/token types

4. Optional Injection with @Optional

4.1 What is @Optional?

The `@Optional` decorator tells Angular that a dependency is optional. If the dependency is not available, Angular injects `null` instead of throwing an error.

Why Use @Optional?

- **Graceful Degradation:** App continues working without optional features
- **Plugin Architecture:** Components work with or without plugins
- **Testing:** Easier to test components without all dependencies
- **Backward Compatibility:** Support older code that might not provide service

4.2 Basic @Optional Example

```
// analytics.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AnalyticsService {
  trackEvent(event: string, data?: any): void {
    console.log('Analytics:', event, data);
  }
}
```

```
// component with optional dependency
import { Component, Optional } from '@angular/core';
import { AnalyticsService } from './analytics.service';

@Component({
  selector: 'app-product',
  template: `
    <div>
      <h3>Product Details</h3>
      <button (click)="viewProduct()">View</button>
      <p>Analytics: {{ hasAnalytics() ? 'Enabled' : 'Disabled' }}</p>
    </div>
  `
})
export class ProductComponent {

  // @Optional makes AnalyticsService optional
  constructor(@Optional() private analytics: AnalyticsService | null) {
    if (this.analytics) {
      console.log('Analytics service available');
    } else {
```

```
        console.log('Analytics service not available - continuing without it');
    }
}

viewProduct(): void {
    // Check if analytics exists before using
    if (this.analytics) {
        this.analytics.trackEvent('product_viewed', { id: 123 });
    }

    // Main functionality works regardless
    console.log('Product viewed');
}

hasAnalytics(): boolean {
    return this.analytics !== null;
}
}
```

Output:

Behavior:

If AnalyticsService is provided:

- analytics is the service instance
- Tracking works normally
- Console: "Analytics service available"

If AnalyticsService is NOT provided:

- analytics is null
- No tracking, but app works
- Console: "Analytics service not available - continuing without it"
- NO ERROR thrown

4.3 Practical Example: Logger Plugin

```
// logger-plugin.service.ts
import { Injectable } from '@angular/core';

@Injectable() // Not provided by default
```

```
export class LoggerPluginService {

  logToFile(message: string): void {
    console.log('[File Logger]', message);
    // In real app: send to server, save to file, etc.
  }

  logToConsole(message: string): void {
    console.log('[Console Logger]', message);
  }
}

// Note: This service is NOT in any providers array initially
```

```
// application.component.ts
import { Component, Optional } from '@angular/core';
import { LoggerPluginService } from './logger-plugin.service';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h2>Application</h2>
      <p>Logger Plugin: {{ pluginStatus() }}</p>
      <button (click)="performAction()">Perform Action</button>
    </div>
  `
})
export class ApplicationComponent {

  constructor(@Optional() private loggerPlugin: LoggerPluginService | null) { }

  performAction(): void {
    const action = 'User clicked button';

    // Use plugin if available
    if (this.loggerPlugin) {
      this.loggerPlugin.logToFile(action);
      this.loggerPlugin.logToConsole(action);
    }

    // Core functionality always works
    console.log('Action performed:', action);
  }
}
```

```
pluginStatus(): string {  
    return this.loggerPlugin ? 'Installed' : 'Not Installed';  
}  
}
```

```
// Enable plugin by providing it in a module  
@NgModule({  
    providers: [LoggerPluginService] // Uncomment to enable plugin  
})  
export class AppModule { }
```

4.4 @Optional with Default Values

```
// Using || operator for defaults  
import { Component, Optional } from '@angular/core';  
import { ConfigService } from './config.service';  
  
@Component({  
    selector: 'app-settings'  
})  
export class SettingsComponent {  
  
    private config: ConfigService;  
  
    constructor(@Optional() configService: ConfigService | null) {  
        // Use provided service or create default  
        this.config = configService || this.createDefaultConfig();  
    }  
  
    private createDefaultConfig(): ConfigService {  
        const defaultConfig = new ConfigService();  
        // Set default values  
        return defaultConfig;  
    }  
}
```

4.5 When to Use @Optional

💡 Tip:

Use @Optional When:

- ✓ Feature is truly optional (app works without it)
- ✓ Building plugin systems
- ✓ Supporting multiple configurations
- ✓ Testing components in isolation
- ✓ Gradual feature rollout

Don't Use When:

- ✗ Dependency is required for component to work
- ✗ Would cause runtime errors if missing
- ✗ Core functionality depends on it

5. @SkipSelf Decorator

5.1 What is @SkipSelf?

`@SkipSelf` tells Angular to skip the current injector and start searching for the dependency in the parent injector. This is useful when you want to use a parent's service instead of creating a new one.

Why Use @SkipSelf?

- **Access Parent's Service:** Child needs parent's specific instance
- **Avoid Recursion:** Prevent service from injecting itself
- **Hierarchy Control:** Force dependency from higher in tree
- **Configuration Inheritance:** Use parent's configuration

5.2 Understanding the Injector Hierarchy

```
// Without @SkipSelf
Component requests Service
↓
Check Component's Injector → Found? → Use it
↓ Not found
Check Parent's Injector → Found? → Use it
↓ Not found
Continue up...
```

```
// With @SkipSelf
Component requests Service with @SkipSelf
↓
SKIP Component's Injector (even if Service is there)
↓
Check Parent's Injector → Found? → Use it
↓ Not found
Continue up...
```

5.3 Basic @SkipSelf Example

```
// counter.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class CounterService {

  private count: number = 0;
  private level: string;

  constructor(level: string) {
    this.level = level;
    console.log(`CounterService created at ${level} level`);
  }

  increment(): void {
    this.count++;
  }

  getCount(): number {
    return this.count;
  }

  getLevel(): string {
    return this.level;
  }
}
```

```
// parent.component.ts
import { Component } from '@angular/core';
import { CounterService } from './counter.service';
```

```
@Component({
  selector: 'app-parent',
  template: `
    <div>
      <h2>Parent Component</h2>
      <p>Level: {{ getLevel() }}</p>
      <p>Count: {{ getCount() }}</p>
      <button (click)="increment()">Increment</button>

      <app-child></app-child>
    </div>
  `,
  providers: [
    {
      provide: CounterService,
      useClass: new CounterService('Parent')
    }
  ]
})
export class ParentComponent {

  constructor(private counter: CounterService) { }

  increment() { this.counter.increment(); }
  getCount() { return this.counter.getCount(); }
  getLevel() { return this.counter.getLevel(); }
}
```

```
// child.component.ts - Using @SkipSelf
import { Component, SkipSelf } from '@angular/core';
import { CounterService } from './counter.service';

@Component({
  selector: 'app-child',
  template: `
    <div style="margin-left: 20px; border-left: 2px solid #ccc; padding: 10px;">
      <h3>Child Component</h3>
      <p>Level: {{ getLevel() }}</p>
      <p>Count: {{ getCount() }}</p>
      <button (click)="increment()">Increment</button>
    </div>
  `,
  providers: [
    {
      provide: CounterService,
```

```
        useValue: new CounterService('Child')
      }
    ]
  })
  export class ChildComponent {

    // @SkipSelf makes it use parent's service, not child's own
    constructor(@SkipSelf() private counter: CounterService) { }

    increment() { this.counter.increment(); }
    getCount() { return this.counter.getCount(); }
    getLevel() { return this.counter.getLevel(); }
  }
```

Output:

Behavior with @SkipSelf:

- Child has its own CounterService in providers
- But @SkipSelf skips child's injector
- Child uses parent's CounterService instead
- Both show: Level: "Parent"
- Both show same count (shared instance)

Without @SkipSelf:

- Child would use its own CounterService
- Child shows: Level: "Child"
- Separate counts (different instances)

5.4 @SkipSelf with @Optional

Combining `@SkipSelf` with `@Optional` handles cases where parent might not provide the service:

```
import { Component, SkipSelf, Optional } from '@angular/core';
import { ThemeService } from '../theme.service';

@Component({
  selector: 'app-child',
  template: '<div>Child</div>'
})
```



```
export class ChildComponent {

  constructor(
    @SkipSelf() @Optional() private parentTheme: ThemeService | null
  ) {
    if (this.parentTheme) {
      console.log('Using parent theme');
    } else {
      console.log('No parent theme available');
    }
  }
}
```

5.5 Preventing Circular Dependencies

@SkipSelf prevents a service from accidentally injecting itself:

```
// hierarchical-data.service.ts
import { Injectable, SkipSelf, Optional } from '@angular/core';

@Injectable()
export class HierarchicalDataService {

  private data: string;

  constructor(
    @SkipSelf() @Optional() private parent: HierarchicalDataService | null
  ) {
    this.data = 'Default';
  }

  getData(): string {
    if (this.parent) {
      // Can access parent's data
      return `${this.parent.getData()} > ${this.data}`;
    }
    return this.data;
  }

  setData(data: string): void {
```

```
this.data = data;  
}  
}
```

Output:

Result:

- Service can safely inject "itself" from parent
- @SkipSelf prevents circular dependency
- Creates hierarchy: Parent → Child → Grandchild
- Useful for tree structures or nested menus

6. Multi-Provider Tokens

6.1 What are Multi-Providers?

Multi-provider tokens allow multiple services to be provided for the same token. When injected, you receive an array of all the provided values. This is useful for plugin systems, hooks, and extensibility.

Why Use Multi-Providers?

- **Plugin Architecture:** Multiple plugins register for same extension point
- **Hook Systems:** Multiple handlers for same event
- **Validators:** Multiple validation rules for same field
- **Interceptors:** Multiple middleware functions

6.2 Creating Multi-Providers

```
// Basic multi-provider syntax  
{  
  provide: TOKEN,  
  useValue: value,  
  multi: true // This is the key!  
}
```

6.3 Simple Multi-Provider Example

```
// logger.interface.ts
export interface Logger {
  log(message: string): void;
}

// console-logger.ts
export class ConsoleLogger implements Logger {
  log(message: string): void {
    console.log('[Console]', message);
  }
}

// alert-logger.ts
export class AlertLogger implements Logger {
  log(message: string): void {
    // In real app, this would show alert
    console.log('[Alert]', message);
  }
}

// file-logger.ts
export class FileLogger implements Logger {
  log(message: string): void {
    console.log('[File]', message);
    // In real app: save to file
  }
}
```

```
// app.module.ts
import { InjectionToken } from '@angular/core';
import { Logger } from './logger.interface';
import { ConsoleLogger } from './console-logger';
import { AlertLogger } from './alert-logger';
import { FileLogger } from './file-logger';

export const LOGGERS = new InjectionToken<Logger[]>('loggers');

@NgModule({
  providers: [
    { provide: LOGGERS, useClass: ConsoleLogger, multi: true },
    { provide: LOGGERS, useClass: AlertLogger, multi: true },
    { provide: LOGGERS, useClass: FileLogger, multi: true }
  ]
})
```

```

    ]
  })
  export class AppModule { }

```

```

// Using multi-provider in a component
import { Component, Inject } from '@angular/core';
import { LOGGERS } from './app.module';
import { Logger } from './logger.interface';

@Component({
  selector: 'app-logging',
  template: `
    <div>
      <h3>Multi-Logger System</h3>
      <p>Active Loggers: {{ getLoggerCount() }}</p>
      <button (click)="logMessage()">Log Message</button>
    </div>
  `,
})
export class LoggingComponent {

  // Receives an ARRAY of all loggers
  constructor(@Inject(LOGGERS) private loggers: Logger[]) {
    console.log(`${this.loggers.length} loggers registered`);
  }

  logMessage(): void {
    const message = 'Important event occurred';

    // Call log() on ALL loggers
    this.loggers.forEach(logger => {
      logger.log(message);
    });
  }

  getLoggerCount(): number {
    return this.loggers.length;
  }
}

```

Output:

Console Output when "Log Message" clicked:

[Console] Important event occurred

[Alert] Important event occurred

[File] Important event occurred

Key Point: One token, multiple implementations. All are called automatically.

6.4 Practical Example: Validation System

```
// validator.interface.ts
export interface Validator {
  validate(value: string): { valid: boolean; error?: string };
}

// email-validator.ts
export class EmailValidator implements Validator {
  validate(value: string): { valid: boolean; error?: string } {
    const isValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
    return {
      valid: isValid,
      error: isValid ? undefined : 'Invalid email format'
    };
  }
}

// length-validator.ts
export class LengthValidator implements Validator {
  validate(value: string): { valid: boolean; error?: string } {
    const isValid = value.length >= 5;
    return {
      valid: isValid,
      error: isValid ? undefined : 'Must be at least 5 characters'
    };
  }
}

// pattern-validator.ts
export class PatternValidator implements Validator {
  validate(value: string): { valid: boolean; error?: string } {
    const isValid = /^[a-zA-Z0-9]+$/.test(value);
    return {
      valid: isValid,
```

```

    error: isValid ? undefined : 'Only letters and numbers allowed'
  };
}
}

```

```

// app.module.ts
import { InjectionToken } from '@angular/core';
import { Validator } from './validator.interface';
import { EmailValidator } from './email-validator';
import { LengthValidator } from './length-validator';
import { PatternValidator } from './pattern-validator';

export const VALIDATORS = new InjectionToken<Validator[]>('validators');

@NgModule({
  providers: [
    { provide: VALIDATORS, useClass: EmailValidator, multi: true },
    { provide: VALIDATORS, useClass: LengthValidator, multi: true },
    { provide: VALIDATORS, useClass: PatternValidator, multi: true }
  ]
})
export class AppModule { }

```

```

// form.component.ts
import { Component, Inject } from '@angular/core';
import { VALIDATORS } from './app.module';
import { Validator } from './validator.interface';

@Component({
  selector: 'app-form',
  template: `
    <div>
      <h3>Registration Form</h3>
      <input [(ngModel)]="inputValue" placeholder="Enter value">
      <button (click)="validateInput()">Validate</button>

      <div *ngIf="errors.length > 0">
        <h4>Errors:</h4>
        <ul>
          <li *ngFor="let error of errors">{{ error }}</li>
        </ul>
      </div>
    </div>
  `
})

```

```
        <p *ngIf="isValid">✓ All validations passed!</p>
    </div>
    `
  })
  export class FormComponent {

    inputValue: string = '';
    errors: string[] = [];
    isValid: boolean = false;

    constructor(@Inject(VARIABLES) private validators: Validator[]) { }

    validateInput(): void {
      this.errors = [];
      this.isValid = true;

      // Run ALL validators
      this.validators.forEach(validator => {
        const result = validator.validate(this.inputValue);
        if (!result.valid) {
          this.isValid = false;
          if (result.error) {
            this.errors.push(result.error);
          }
        }
      });
    }
  }
}
```

Output:

Input: "abc"

Errors:

- Invalid email format
- Must be at least 5 characters

Input: "abc@def"

Errors:

- Invalid email format

Input: "user123@email.com"

✓ All validations passed!

6.5 Adding Validators Dynamically

Different modules can add their own validators:

```
// feature.module.ts
import { NgModule } from '@angular/core';
import { VALIDATORS } from './app.module';
import { CustomValidator } from './custom-validator';

@NgModule({
  providers: [
    // Add another validator to the multi-provider
    { provide: VALIDATORS, useClass: CustomValidator, multi: true }
  ]
})
export class FeatureModule { }
```

⚡ Important:

Multi-Provider Benefits:

- Extensible: Easy to add new implementations
- Decoupled: Implementations don't know about each other
- Modular: Different modules can contribute
- Type-safe: Array of typed interfaces

6.6 When to Use Multi-Providers

💡 Tip:

Use Multi-Providers When:

- ✓ Need plugin/extension system
- ✓ Multiple handlers for same event
- ✓ Collecting implementations from different modules

- ✓ Building validation/middleware chains
- ✓ Framework-style extensibility points

Don't Use When:

- ✗ Only one implementation needed
- ✗ Simple service injection works fine
- ✗ Services don't share common interface

7. Summary

⚡ **Important:**

Key Takeaways:

1. Factory Providers:

- Use functions to create services
- Pass configuration or dependencies to factory
- Choose implementations based on conditions
- Syntax: `useFactory` with `deps` array

2. `useValue` Provider:

- Provide existing objects or constants
- Perfect for configuration objects
- Use `InjectionToken` for type safety
- No service creation involved

3. `useExisting` Provider:

- Create aliases for services
- Multiple tokens for same instance
- Useful for abstraction and backward compatibility
- References existing provider

4. `@Optional` Decorator:

- Makes dependencies optional
- Returns null if not provided

- App continues without optional services
- Great for plugins and feature flags

5. @SkipSelf Decorator:

- Skip current injector, use parent's
- Access parent's service instance
- Prevent circular dependencies
- Build hierarchical services

6. Multi-Provider Tokens:

- Multiple services for same token
- Injected as array
- Set `multi: true` in provider
- Perfect for plugin systems

7. When to Use:

- Factory: Configuration, conditional logic
- `useValue`: Constants, existing objects
- `useExisting`: Aliases, abstraction
- `@Optional`: Optional features
- `@SkipSelf`: Hierarchy, parent access
- Multi: Plugins, extensibility

8. Using Advanced DI with Standalone Components

⚡ Important:

Good News! Every Advanced DI technique shown in this module works perfectly with standalone components. You don't need NgModules to use factory providers, `useValue`, `useExisting`, `@Optional`, `@SkipSelf`, or multi-providers.

8.1 Provider Configuration in Standalone

In standalone components, providers can be configured in three main ways:

Location	Scope	Use Case
Component providers	Component and children	Component-specific services
bootstrapApplication	Entire application	Root-level services
Route providers	Route and children	Feature-specific services

8.2 Standalone: Factory Provider Example

```
// logger.service.ts
export class LoggerService {
  constructor(private prefix: string) { }

  log(message: string): void {
    console.log(`${this.prefix}: ${message}`);
  }
}

// logger.factory.ts
export function loggerFactory(): LoggerService {
  const environment = 'development';
  const prefix = environment === 'development' ? '[DEV]' : '[PROD]';
  return new LoggerService(prefix);
}

// app.component.ts - Standalone Component
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { LoggerService } from './logger.service';
import { loggerFactory } from './logger.factory';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule],
  providers: [
    // Factory provider in standalone component
    {
      provide: LoggerService,
      useFactory: loggerFactory
    }
  ],
  template: `
```

```
<h1>Standalone App</h1>
<button (click)="testLogger()">Test Logger</button>
`
})
export class AppComponent {
  constructor(private logger: LoggerService) { }

  testLogger(): void {
    this.logger.log('Factory provider in standalone component!');
  }
}
```

Output:

Console when button clicked:

[DEV]: Factory provider in standalone component!

8.3 Standalone: Application-Level Providers

For application-wide providers, use `bootstrapApplication`:

```
// main.ts - Application Bootstrap
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { ConfigService } from './app/config.service';
import { HttpClientService } from './app/http-client.service';
import { httpClientFactory } from './app/http-client.factory';

bootstrapApplication(AppComponent, {
  providers: [
    // Factory provider with dependencies
    ConfigService,
    {
      provide: HttpClientService,
      useFactory: httpClientFactory,
      deps: [ConfigService]
    }
  ]
}).catch(err => console.error(err));
```

8.4 Standalone: useValue with InjectionToken

```
// app.config.ts
import { InjectionToken } from '@angular/core';

export interface AppConfig {
  apiUrl: string;
  appName: string;
  version: string;
}

export const APP_CONFIG_TOKEN = new InjectionToken<AppConfig>('app.config');

export const APP_CONFIG: AppConfig = {
  apiUrl: 'https://api.example.com',
  appName: 'Standalone App',
  version: '2.0.0'
};

// settings.component.ts - Standalone
import { Component, Inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { APP_CONFIG_TOKEN, AppConfig, APP_CONFIG } from './app.config';

@Component({
  selector: 'app-settings',
  standalone: true,
  imports: [CommonModule],
  providers: [
    // useValue provider in standalone
    {
      provide: APP_CONFIG_TOKEN,
      useValue: APP_CONFIG
    }
  ],
  template: `
    <h3>{{ config.appName }}</h3>
    <p>Version: {{ config.version }}</p>
    <p>API: {{ config.apiUrl }}</p>
  `
})
export class SettingsComponent {
  constructor(@Inject(APP_CONFIG_TOKEN) public config: AppConfig) { }
}
```

8.5 Standalone: @Optional and @SkipSelf

Decorators work exactly the same in standalone components:

```
// analytics.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Available everywhere
})
export class AnalyticsService {
  trackEvent(event: string): void {
    console.log('Analytics:', event);
  }
}

// product.component.ts - Standalone with @Optional
import { Component, Optional } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AnalyticsService } from './analytics.service';

@Component({
  selector: 'app-product',
  standalone: true,
  imports: [CommonModule],
  template: `
    <button (click)="viewProduct()">View Product</button>
    <p>Analytics: {{ hasAnalytics() ? 'Enabled' : 'Disabled' }}</p>
  `
})
export class ProductComponent {
  // @Optional works in standalone
  constructor(@Optional() private analytics: AnalyticsService | null) { }

  viewProduct(): void {
    if (this.analytics) {
      this.analytics.trackEvent('product_viewed');
    }
    console.log('Product viewed');
  }

  hasAnalytics(): boolean {
```

```
    return this.analytics !== null;
  }
}
```

8.6 Standalone: Multi-Provider Example

```
// logger.interface.ts
export interface Logger {
  log(message: string): void;
}

// console-logger.ts
export class ConsoleLogger implements Logger {
  log(message: string): void {
    console.log('[Console]', message);
  }
}

// file-logger.ts
export class FileLogger implements Logger {
  log(message: string): void {
    console.log('[File]', message);
  }
}

// main.ts - Multi-providers at application level
import { bootstrapApplication } from '@angular/platform-browser';
import { InjectionToken } from '@angular/core';
import { AppComponent } from './app/app.component';
import { Logger } from './app/logger.interface';
import { ConsoleLogger } from './app/console-logger';
import { FileLogger } from './app/file-logger';

export const LOGGERS = new InjectionToken<Logger[]>('loggers');

bootstrapApplication(AppComponent, {
  providers: [
    // Multi-providers in standalone
    { provide: LOGGERS, useClass: ConsoleLogger, multi: true },
    { provide: LOGGERS, useClass: FileLogger, multi: true }
  ]
});
```

```
// logging.component.ts - Using multi-provider
import { Component, Inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { LOGGERS } from '../main';
import { Logger } from './logger.interface';

@Component({
  selector: 'app-logging',
  standalone: true,
  imports: [CommonModule],
  template: `
    <h3>Multi-Logger System</h3>
    <p>Active Loggers: {{ loggers.length }}</p>
    <button (click)="logMessage()">Log Message</button>
  `
})
export class LoggingComponent {
  constructor(@Inject(LOGGERS) public loggers: Logger[]) { }

  logMessage(): void {
    this.loggers.forEach(logger => {
      logger.log('Message from standalone component');
    });
  }
}
```

8.7 Standalone: Component-Level Providers

Providers in standalone components create instances scoped to that component:

```
// counter.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class CounterService {
  private count = 0;

  increment(): void { this.count++; }
  getCount(): number { return this.count; }
}

// parent.component.ts - Standalone
import { Component } from '@angular/core';
```



```
import { CommonModule } from '@angular/common';
import { CounterService } from '../counter.service';
import { ChildComponent } from '../child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [CommonModule, ChildComponent],
  providers: [CounterService], // Provided at component level
  template: `
    <h2>Parent: {{ counter.getCount() }}</h2>
    <button (click)="counter.increment()">Increment</button>
    <app-child></app-child>
  `
})
export class ParentComponent {
  constructor(public counter: CounterService) { }
}

// child.component.ts - Standalone with @SkipSelf
import { Component, SkipSelf } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CounterService } from '../counter.service';

@Component({
  selector: 'app-child',
  standalone: true,
  imports: [CommonModule],
  providers: [CounterService], // Has its own, but will skip it
  template: `
    <h3>Child: {{ counter.getCount() }}</h3>
    <button (click)="counter.increment()">Increment</button>
  `
})
export class ChildComponent {
  // @SkipSelf works in standalone
  constructor(@SkipSelf() public counter: CounterService) { }
}
```

Output:

Result:

- Both parent and child share the same counter
- Clicking either button increments the shared count
- @SkipSelf makes child use parent's provider
- Works identically to NgModule-based approach

8.8 Standalone: Route-Level Providers

Standalone routing allows providers at the route level:

```
// app.routes.ts
import { Routes } from '@angular/router';
import { DashboardComponent } from '../dashboard/dashboard.component';
import { DashboardService } from '../dashboard/dashboard.service';

export const routes: Routes = [
  {
    path: 'dashboard',
    component: DashboardComponent,
    providers: [
      // Providers scoped to this route and its children
      DashboardService,
      {
        provide: 'DashboardConfig',
        useValue: { refreshInterval: 5000 }
      }
    ]
  }
];

// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes) // Routes with their providers
  ]
});
```

8.9 Migration: NgModule to Standalone

Converting Advanced DI from NgModule to Standalone is straightforward:

NgModule Approach	Standalone Approach
Module <code>providers</code> array	<code>bootstrapApplication</code> <code>providers</code>
Component in <code>declarations</code>	Component with <code>standalone: true</code>
Module <code>imports</code>	Component <code>imports</code> array
Lazy-loaded modules	Lazy-loaded routes with providers

Before (NgModule):

```
@NgModule({
  providers: [
    {
      provide: LoggerService,
      useFactory: loggerFactory
    }
  ]
})
export class AppModule { }
```

After (Standalone):

```
bootstrapApplication(AppComponent, {
  providers: [
    {
      provide: LoggerService,
      useFactory: loggerFactory
    }
  ]
});
```

💡 **Tip:**

Best Practices for Standalone Components:

- ✓ Use `providedIn: 'root'` for singleton services
- ✓ Configure root providers in `bootstrapApplication`
- ✓ Use component providers for component-scoped services
- ✓ Use route providers for feature-scoped services
- ✓ All decorators (`@Optional`, `@SkipSelf`) work identically
- ✓ Factory, `useValue`, `useExisting`, multi-providers all supported
- ✓ Type safety with `InjectionToken` remains the same
- ✓ Hierarchical DI works exactly the same way

8.10 Complete Standalone Example

Here's a complete standalone application using all Advanced DI techniques:

```
// main.ts - Complete standalone setup
import { bootstrapApplication } from '@angular/platform-browser';
import { InjectionToken } from '@angular/core';
import { AppComponent } from './app/app.component';

// Configuration
export interface AppConfig {
  apiUrl: string;
  debug: boolean;
}

export const APP_CONFIG_TOKEN = new InjectionToken<AppConfig>('app.config');

const config: AppConfig = {
  apiUrl: 'https://api.example.com',
  debug: true
};

// Factory
export function httpClientFactory(config: AppConfig) {
  return new HttpClientService(config.apiUrl, config.debug);
}

// Multi-provider
export interface Plugin {
  init(): void;
}
```

```
export const PLUGINS = new InjectionToken<Plugin[]>('plugins');

class AnalyticsPlugin implements Plugin {
  init() { console.log('Analytics initialized'); }
}

class LoggerPlugin implements Plugin {
  init() { console.log('Logger initialized'); }
}

// Bootstrap with all providers
bootstrapApplication(AppComponent, {
  providers: [
    // useValue
    { provide: APP_CONFIG_TOKEN, useValue: config },

    // Factory with deps
    ConfigService,
    {
      provide: HttpClientService,
      useFactory: httpClientFactory,
      deps: [{ token: APP_CONFIG_TOKEN }]
    },

    // Multi-providers
    { provide: PLUGINS, useClass: AnalyticsPlugin, multi: true },
    { provide: PLUGINS, useClass: LoggerPlugin, multi: true }
  ]
}).catch(err => console.error(err));
```

⚡ Important:

Key Insight: Standalone components don't change how Advanced DI works - they just change **where** you configure providers. The DI system, decorators, and provider types remain identical. Whether you use NgModules or standalone components is a architectural choice, not a limitation on DI features.

9. Practice Exercise

 Note:

Exercise: Advanced DI Plugin System

Build a notification system using all advanced DI techniques:

1. Factory Provider for NotificationService:

- Create factory that configures service based on environment
- Development: verbose logging, all notifications
- Production: minimal logging, important only
- Factory depends on ConfigService

2. useValue for Configuration:

- Create AppConfig interface with environment, debugMode, maxNotifications
- Use InjectionToken for type safety
- Provide different configs for dev/prod

3. Multi-Provider for Notification Handlers:

- Create NotificationHandler interface with handle(message) method
- Implement: ConsoleHandler, ToastHandler, EmailHandler
- Register all as multi-providers
- NotificationService loops through all handlers

4. @Optional for AnalyticsService:

- Create optional AnalyticsService
- NotificationService uses it if available
- Tracks notification metrics

5. @SkipSelf for Parent Preferences:

- Create UserPreferencesService
- Child components use parent's preferences with @SkipSelf
- Preferences include: enableNotifications, soundEnabled

6. Component Requirements:

- NotificationComponent - trigger notifications
- PreferencesComponent - manage user preferences
- Show handler count, analytics status
- Allow enabling/disabling handlers

7. Test Scenarios:

- Send notification → all handlers execute
- Disable AnalyticsService → app works without it
- Child inherits parent preferences
- Different configs for dev/prod

8. Bonus Challenge:

- Implement the entire system using **standalone components**
- Configure providers in bootstrapApplication and component-level
- Use route-level providers for feature-specific services
- Compare the standalone approach with NgModule approach

Congratulations! You've completed the Services and Dependency Injection module. Next: Module 7 - Pipes and Data Transformation